MODULE 19 · DOMAIN 2 - CONTAINER ORCHESTRATION

Network policy, secrets and the supply chain

Limiting traffic, protecting stored data, watching what runs, and trusting what you deploy.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 19 of 35

admin@sthithapragna.com

Defence in depth: limit who can talk, protect what is stored, watch what runs, trust what you deploy.

NetworkPolicy: the defaults

- **By default, every Pod can reach every other Pod,** in every namespace.
- **Once a policy selects a Pod,** that direction becomes isolated: only allowed traffic gets through.
- **Policies add up.** The allowed traffic is the union of all policies. There are no deny rules.
- Namespaced, layer 3 and 4 only, and enforced by the network plugin, as module 13 showed.

> ON THE EXAM
>
> No policy selects a Pod: everything allowed. Any policy selects it: only what policies allow.

Choosing who is allowed

SELECTOR	ALLOWS TRAFFIC FROM, OR TO...
podSelector	Pods with these labels, in the policy's own namespace
namespaceSelector	All Pods in namespaces with these labels
Both, in one entry	Pods with these labels, in namespaces with those labels
ipBlock	A CIDR range, typically outside the cluster
ports	Which ports and protocols, such as TCP 5432

ON THE EXAM

Policies select Pods by label, never by IP. Pod IPs change; labels do not.

Step 1: deny all incoming traffic

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-ingress, namespace: quayside}
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

5 An empty selector selects every Pod in the namespace.

6 Ingress, with no rules: nothing is allowed in.

THE TAKEAWAY

Start from default-deny, then allow each flow you need. Anything not written down is blocked.

Step 2: let only the booking API in

```
1  apiVersion: networking.k8s.io/v1
2  kind: NetworkPolicy
3  metadata: {name: db-allow-booking-api, namespace: quayside}
4  spec:
5    podSelector: {matchLabels: {app: bookings-db}}
6    policyTypes: [Ingress]
7    ingress:
8    - from:
9      - podSelector: {matchLabels: {app: booking-api}}
10     ports: [{protocol: TCP, port: 5432}]
```

5 This policy protects the database Pods.

9-10 Only booking API Pods, and only on the Postgres port.

The dash that changes everything

TWO ENTRIES: OR

Wider than it looks

```
- from:
  - namespaceSelector:
      matchLabels: {team: payments}
  - podSelector:
      matchLabels: {app: worker}
```

Any Pod in payments namespaces, or worker Pods in this one.

ONE ENTRY: AND

What was meant

```
- from:
  - namespaceSelector:
      matchLabels: {team: payments}
    podSelector:
      matchLabels: {app: worker}
```

Only worker Pods, in payments namespaces.

THE TIEBREAKER

One dash: AND. Two dashes: OR. A single extra dash can open a policy right up.

Egress policies, and the DNS trap

- **Egress rules** limit where Pods may send traffic: other Pods, or ipBlock ranges.
- **Default-deny egress also blocks DNS.** Every name lookup fails.
- **Allow port 53, UDP and TCP,** to CoreDNS in kube-system, alongside real destinations.
- Symptom to recognise: 'could not resolve host', even for in-cluster Services.

> ON THE EXAM
>
> Egress locked down and name lookups fail: allow DNS to kube-dns on port 53.

What NetworkPolicy cannot do

- **No layer 7:** it sees IPs and ports, not HTTP paths or methods. Meshes and some plugins go further.
- **No encryption:** it restricts traffic but never encrypts it. That is mTLS, module 16.
- **No effect without a supporting plugin,** as module 13 showed.
- Cluster-wide administrator policies are being developed by SIG Network.

> ON THE EXAM
>
> NetworkPolicy restricts. mTLS encrypts and authenticates. Strong setups use both.

Base64 is not protection. Encrypt Secrets at rest, and record who touched what.

Encryption at rest: providers

PROVIDER	KEY HELD	NOTE
`identity`	None	The default: no encryption at all
`aescbc, aesgcm, secretbox`	A key file on the control plane	Encrypts, but the key sits beside the data
`kms (v2)`	An external key management service	Envelope encryption. Recommended

ON THE EXAM

Set in an EncryptionConfiguration passed to the API server. Existing Secrets are encrypted when next written.

Audit logging

- **The API server can log every request:** who, what, when, and the result.
- **An audit policy sets the detail per resource:** None, Metadata, Request, RequestResponse.
- **Log Secrets at Metadata only.** Logging their bodies would copy the secrets into logs.
- **Ship logs off the cluster,** so an attacker cannot erase their tracks.

> ON THE EXAM
>
> 'Who deleted that Deployment at 3am?' is answered by the API server's audit log.

Prevention is never perfect. **Runtime security watches what containers actually do.**

Falco

- **A CNCF graduated project** for runtime threat detection.
- **Watches system calls** through the kernel, usually with eBPF.
- **Rules flag suspicious behaviour:** a shell opened in a container, a write under /etc, a sensitive file read.
- **It detects and alerts;** it does not block by default. Tools like Tetragon can also enforce.

> ON THE EXAM
>
> 'Alert when someone opens a shell in a production container': runtime security, such as Falco.

The software supply chain

01
Source
Code review; dependency scanning.

02
Build
Trusted CI; provenance recorded.

03
Image
SBOM generated; vulnerabilities scanned.

04
Registry
Image signed; trusted registry only.

05
Admission
Signature and source verified before running.

ON THE EXAM

Each stage adds evidence. Admission is where the cluster checks that evidence.

Supply chain terms and tools

TERM	MEANS	EXAMPLES
SBOM	Every component inside an image	SPDX, CycloneDX
Scanning	Components checked against known CVEs	Trivy, Grype
Signing	Who built it, and that it is unchanged	Sigstore cosign, Notary
Provenance	How and where it was built	SLSA levels
Trusted registry	Stores, scans and serves signed images	Harbor (CNCF graduated)

ON THE EXAM

New CVE announced? An SBOM shows in minutes which images contain it.

cert-manager: certificates as Kubernetes objects

- **A CNCF graduated project** that issues and renews TLS certificates.
- **Issuers** connect to a source: ACME services such as Let's Encrypt, Vault, or a private CA.
- **A Certificate object** asks for a certificate; the result lands in a kubernetes.io/tls Secret.
- **Renewal is automatic,** so certificates on Ingresses and Gateways never quietly expire.

> ON THE EXAM
>
> 'TLS certificates that renew themselves inside the cluster': cert-manager.

> Quayside applied a default-deny ingress policy to the quayside namespace. Now the web front end cannot reach the booking API.
>
> Only web Pods should be able to reach the booking API, and only on port 8080.

What should the team add?

A Delete the default-deny policy so traffic flows again

B An egress policy on web Pods allowing traffic to the API

C An ingress policy on booking API Pods allowing web on 8080

D Change the booking API Service to type NodePort

◆ PAUSE AND DECIDE

What should the team add?

A Delete the default-deny policy so traffic flows again

Opens everything, including the database, to everyone.

B An egress policy on web Pods allowing traffic to the API

Egress was never blocked. The API's ingress still denies it.

C An ingress policy on booking API Pods allowing web on 8080

Selects the receiver and allows exactly one flow in.

D Change the booking API Service to type NodePort

Service types do not bypass NetworkPolicy.

THE SIGNAL

Incoming traffic blocked: add an ingress rule on the receiving Pods.

> The team applies a default-deny egress policy to the payments namespace, then adds a rule allowing traffic to booking API Pods.
>
> Every call from the payment worker now fails with: could not resolve host booking-api.quayside.

What is wrong?

A CoreDNS crashed when the policy was applied

B DNS traffic is blocked; allow port 53 to kube-dns

C kube-proxy cannot route with egress policies

D Egress policies are not supported by Kubernetes

◆ PAUSE AND DECIDE

What is wrong?

A CoreDNS crashed when the policy was applied
CoreDNS is fine; the worker just cannot reach it.

B DNS traffic is blocked; allow port 53 to kube-dns
Default-deny egress blocked lookups. Allow UDP and TCP 53.

C kube-proxy cannot route with egress policies
The failure is resolving the name, before any routing.

D Egress policies are not supported by Kubernetes
They are supported, and are working all too well.

THE SIGNAL

'Could not resolve host' right after default-deny egress: allow DNS.

> Quayside's security team wants an alert whenever someone opens an interactive shell inside a production container.
>
> They also want alerts when a container reads sensitive files such as /etc/shadow.

Which kind of tool fits?

A Pod Security Admission at the Restricted level

B A default-deny NetworkPolicy in production

C Vulnerability scanning of images in CI

D Runtime threat detection, such as Falco

◆ PAUSE AND DECIDE

Which kind of tool fits?

A Pod Security Admission at the Restricted level

Checks Pods when created; sees nothing that happens later.

B A default-deny NetworkPolicy in production

Controls network traffic, not processes or file reads.

C Vulnerability scanning of images in CI

Finds known flaws before deployment, not live behaviour.

D Runtime threat detection, such as Falco

Watches system calls and alerts on exactly these behaviours.

THE SIGNAL

Alert on behaviour inside running containers: runtime security.

> Quayside wants only images built by its own CI system to run in production.
>
> When a new vulnerability is announced, the team also wants to know within minutes which running images contain the affected library.

Which combination meets both needs?

A Sign images and make SBOMs in CI; verify at admission

B Scan all running images for vulnerabilities once a month

C Enforce Pod Security Restricted on production namespaces

D Always deploy the latest tag so images stay up to date

◆ PAUSE AND DECIDE

Which combination meets both needs?

A Sign images and make SBOMs in CI; verify at admission

Signatures prove the source; SBOMs answer 'which images?' fast.

B Scan all running images for vulnerabilities once a month

Slow, and says nothing about who built the images.

C Enforce Pod Security Restricted on production namespaces

Controls Pod privileges, not image origin.

D Always deploy the latest tag so images stay up to date

Makes origin and content less certain, not more.

THE SIGNAL

Trusted origin plus fast inventory: signing, SBOMs, and admission verification.

Module 19 in five lines

- **All traffic is allowed** until a policy selects a Pod; then only what policies allow. No deny rules.
- Start with **default-deny**, allow each flow. One dash AND, two dashes OR. **Allow DNS** for egress.
- **Encrypt Secrets at rest,** ideally with KMS v2; **audit log** who did what.
- **Falco** detects suspicious behaviour at runtime, from system calls.
- **Supply chain:** SBOMs, scanning, signing, verified at admission. **cert-manager** for TLS.

Security covered. Next: storage, and how data outlives the Pods that write it.

admin@sthithapragna.com